FLAG

FLAG · MÓDULO 03 · SESSÃO 1

Git: o controlo de versões

Porque é que os developers não guardam "final-v2-AGORA-SIM". Instalar e configurar o Git, e os primeiros commits.

4.ª feira, 11 nov 2026 · 9:30–12:30 · MS Teams · Workshop: Versões de Código com GitHub (1 de 3)

FLAG

Boas práticas Live Training



Escolher um sítio calmo, sem distrações



Ser pontual



Desligar microfone. Ligar quando for para intervir



Manter a câmara ligada



Levantar a mão quando pretender participar



No intervalo, desligar a câmara e o microfone



“Talk is cheap. Show me the code.”

Linus Torvalds, criador do Linux e do Git

Quem vos acompanha



João Gonçalves

Interactive Developer · Trainer · Consultant

Licenciatura em Matemática Aplicada
Mestrado em Comunicação Multimédia
Formação e consultoria em front-end: Angular, React, JavaScript

Contactos

joaogoncalves.net · @joaopapin
edu@joaogoncalves.net

“Making the complicated simple.”

FLAG

RONDA RÁPIDA

E vocês?

1

Nome e de onde vêm

Área de formação ou profissão atual.

2

Experiência

Já usaram o terminal? Já ouviram falar de Git? “Nada” é uma ótima resposta.

3

Porquê este módulo

Curso completo ou só Git e GitHub? O que vos trouxe?

4

Uma expectativa

O que querem saber fazer no fim do módulo, a 16 de novembro?

Um minuto cada, de microfone aberto

Quem está no curso desde setembro: só o 1 e o 4. Quem chega agora: os quatro.

As 3 sessões deste módulo

1

Git, no vosso computador

4.ª, 11 nov (hoje). O controlo de versões, instalar e configurar, os primeiros commits, ver e desfazer.

2

Branches e merge

6.ª, 13 nov. Experimentar sem estragar: branches, merge e conflitos, o .gitignore e voltar atrás.

3

O GitHub

2.ª, 16 nov. O repositório na nuvem: push, pull e clone, trabalhar em equipa (Pull Requests) e publicar um site.

No fim do módulo

Um projeto vosso no GitHub, com o histórico todo, e um site publicado com o GitHub Pages. E o hábito de guardar o trabalho com o Git, para todos os projetos.

FLAG

COMO TRABALHAMOS

O ritmo de cada sessão

1

Teoria curta

Só o essencial, com desenhos e exemplos reais. Os slides ficam convosco no fim.

2

Fazemos juntos

Paro e escrevemos os comandos em conjunto, cada um no seu terminal. Partilham o ecrã quando bloquearem.

3

Exercícios

Vocês resolvem sozinhos, com tempo marcado. O enunciado está no slide e no GitHub.

4

Solução

Corrigimos sempre juntos, no slide seguinte. Errar aqui é o objetivo.

Uma pasta de trabalho só vossa: flag-frontend

Hoje, `flag-frontend/03-git-github/s01`. Lá dentro, cada projeto é um repositório (o livro, as receitas); a `s01` não. Guardem-na: o livro continua na 6.ª.

ONDE ESTÁ O MATERIAL

Três sítios, nada mais

1

GitHub

github.com/JoaoGoncalves/flag-frontend-2026
Pasta 03-git-github: snippets, enunciados e soluções.

2

Moodle FLAG

Fórum do módulo Git e GitHub: entregas dos exercícios e trabalhos de casa.

3

MS Teams

As sessões em direto, o chat durante a aula e avisos rápidos.

Dúvidas fora da aula: chat do Teams ou WhatsApp +351 910 346 437

Respondo no próprio dia útil. Para o Git, mandem o comando que escreveram e a resposta do terminal (copiada ou print).

HOJE

O plano da sessão

1

Controlo de versões

O problema dos "final-v2", o que é um controlo de versões e as filosofias. O Git.

2

Instalar e configurar

O terminal em 5 comandos, o Git instalado e configurado (o nome e o email).

3

O primeiro repositório

As três áreas e os comandos init, status, add, commit e log, num livro.

4

Ver e desfazer

O diff, o restore e as boas mensagens. Depois, o site das receitas no Git, sozinhos.

ANTES DE COMEÇAR: NO TERMINAL

```
$ git --version
git version 2.51.0
```

Sem Git? Não há problema

Instalamos juntos na parte 2. Hoje não há nada para descarregar do GitHub: começamos do zero. Pausa às 10:50.

FLAG

PARTE 1

Controlo de versões

guardar cada versão do trabalho, e poder voltar atrás

O PROBLEMA → O QUE É → AS FILOSOFIAS → O GIT

FLAG

TEORIA

O problema: qual é a versão boa?



Qual é a última?

E qual é a que funcionava? O que mudou entre a "final" e a "final-v2"?

E se forem três pessoas?

Cada uma com a sua cópia, por email ou numa pen. Quem junta tudo, e como?

E num site com 200 ficheiros?

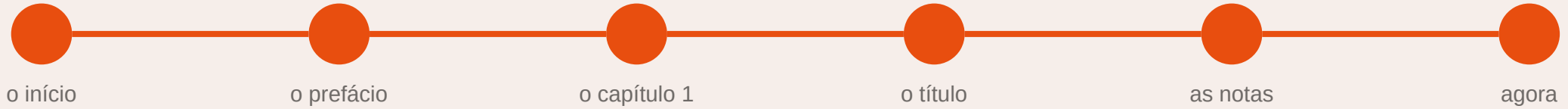
Copiar a pasta inteira de cada vez? Não se aguenta.

FLAG

TEORIA

O que é um controlo de versões

O histórico do projeto: cada versão guardada é um ponto de gravação (como num jogo)



1

Guarda cada versão

De todos os ficheiros do projeto, com quem, quando e porquê.

2

Volta atrás

A qualquer versão anterior, de um ficheiro ou do projeto todo.

3

Compara

O que mudou entre duas versões, linha a linha.

4

Trabalho em equipa

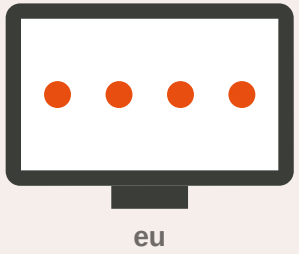
Cada um no seu computador; depois, junta-se o trabalho de todos.

FLAG

TEORIA

As filosofias: local, centralizado, distribuído

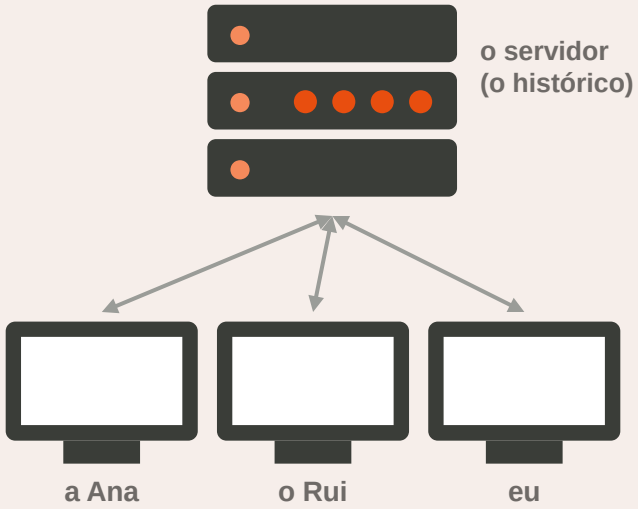
Local



O histórico só no meu computador. Sem equipa.

RCS (1982)

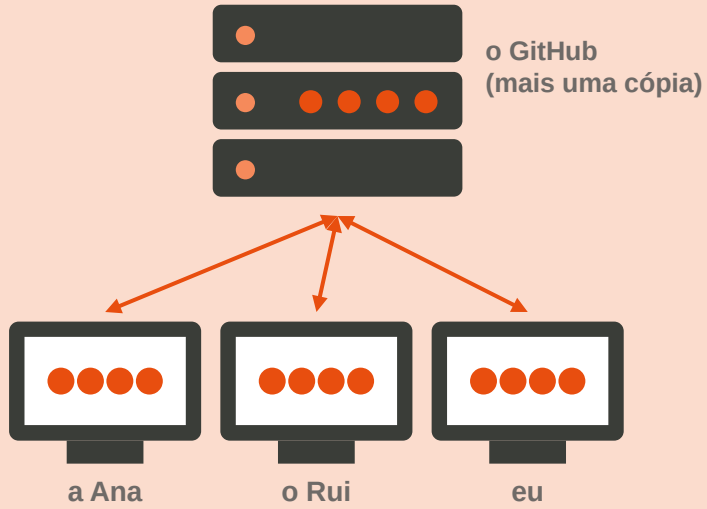
Centralizado



Um servidor tem o histórico; cada um tem só a última versão. Sem servidor, não há commits.

CVS, Subversion (SVN)

Distribuído



Cada computador tem o histórico completo. Trabalha-se offline; o servidor é só mais uma cópia.

Git, Mercurial

FLAG

TEORIA

O Git (e o que o Git não é)

2005 · Linus Torvalds

O criador do Linux precisava de gerir o trabalho de milhares de programadores no mesmo projeto. Não gostou de nenhum dos que havia: escreveu o seu, em poucas semanas.

Hoje: é o controlo de versões de quase todos os projetos de software do mundo (o VS Code, o React, o Linux…).

1 Distribuído

O histórico todo no vosso computador: rápido e offline.

2 Seguro

Cada commit tem uma "impressão digital" (o hash).

3 Branches baratos

Experimentar sem estragar o que funciona (6.ª).

4 Gratuito e open source

E com GitHub, GitLab e Bitbucket à volta.

Git ≠ GitHub

O Git é o programa, no vosso computador: guarda o histórico.

O GitHub é um site: guarda uma cópia do repositório na nuvem e tem as ferramentas de equipa.

O Git trabalha sem GitHub. O GitHub é na 2.ª feira.

FLAG

PARTE 2

Instalar e configurar

o terminal, o Git instalado e a nossa identidade

O TERMINAL → INSTALAR → CONFIGURAR → JUNTOS

FLAG

TEORIA

O terminal em 5 comandos

TERMINAL · MAC, LINUX E GIT BASH (WINDOWS)

```
$ pwd  # onde estou?
/Users/ana/Documents

$ ls  # o que há aqui?
flag-frontend   receitas   fotos

$ cd flag-frontend  # entrar na pasta
$ cd ..  # voltar à de cima

$ mkdir s01  # criar uma pasta
$ clear  # limpar o ecrã
```

Onde o abrir

Mac: a aplicação Terminal. Windows: o Git Bash (vem com o Git). No VS Code: Ctrl+ (Terminal › New Terminal).

Dicas

A tecla Tab completa os nomes. A seta ↑ repete o último comando. Ctrl+C cancela.

No VS Code, sem terminal?

O VS Code também faz Git com botões (o separador Source Control). Mas primeiro aprendemos os comandos: percebe-se o que os botões fazem, e funciona em qualquer lado.

FLAG

TEORIA

Instalar o Git

1

Windows

git-scm.com › Download for Windows. Next, Next… com duas escolhas: o editor (Visual Studio Code) e o nome do ramo inicial, "main". Fica o Git Bash.

2

Mac

Normalmente já vem (ou pede para instalar as "Command Line Tools" à primeira vez). Ou: brew install git.

3

Linux

sudo apt install git (Ubuntu/Debian) ou o gestor de pacotes da distribuição.

E CONFIRMAR

```
$ git --version
git version 2.51.0
```

Fechar e abrir o terminal

Depois de instalar, o terminal (e o VS Code) só "veem" o Git quando se abrem outra vez.

FLAG

TEORIA

Quem sou eu? A configuração

UMA VEZ POR COMPUTADOR

```
# o nome e o email ficam em todos os commits
$ git config --global user.name "Ana Silva"
$ git config --global user.email "ana@exemplo.pt"

# o ramo inicial chama-se main
$ git config --global init.defaultBranch main

# o editor para as mensagens: o VS Code
$ git config --global core.editor "code --wait"

# ver tudo
$ git config --list
```

--global

Vale para todos os repositórios deste computador. Faz-se uma vez.

O email

Usem o mesmo que vão usar no GitHub (2.ª feira): é assim que ele vos reconhece.

E a autenticação?

Entrar no GitHub a partir do Git: na sessão 3.

FLAG

Configurar o Git de toda a gente

1. Terminal aberto (Git Bash no Windows) e `git --version` a responder
2. `git config --global user.name "o vosso nome"`
3. `git config --global user.email "o vosso email"`
4. `git config --global init.defaultBranch main`
5. `git config --global core.editor "code --wait"`
6. `git config --list`: copiem as linhas user.name e user.email para o chat

PARTE 3

O primeiro repositório

as três áreas do Git e os primeiros commits

AS TRÊS ÁREAS → OS COMANDOS → O LIVRO, JUNTOS

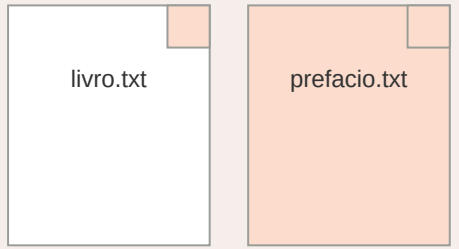
FLAG

TEORIA

As três áreas do Git

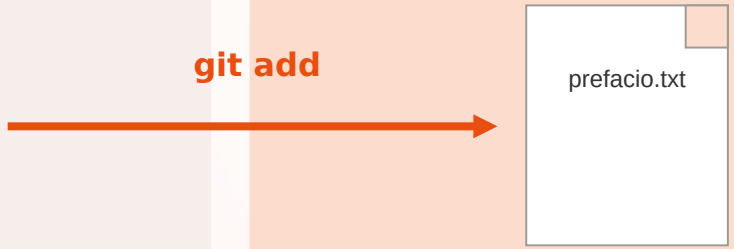
Working directory

a pasta onde trabalhamos: os ficheiros que vemos e editamos



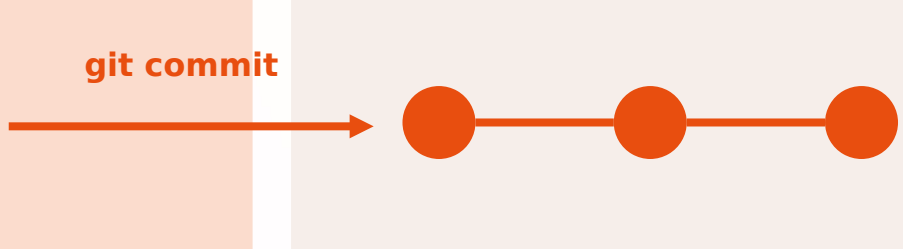
Staging area

a "caixa" do próximo commit: o que escolhemos guardar



Repositório (.git)

o histórico: todos os commits, guardados para sempre



1

Modificado

Mudei o ficheiro, mas o Git ainda não o vai guardar.

2

Preparado (staged)

Escolhi-o com o git add: vai no próximo commit.

3

Guardado (committed)

Está no histórico. Posso voltar a ele quando quiser.

FLAG

TEORIA

Os cinco comandos do dia a dia

1. **git init**
   Uma vez, na pasta do projeto: cria o repositório (a pasta .git).

2. **git status**
   Sempre que houver dúvidas: o que mudou e o que está na caixa.

3. **git add ficheiro**
   Pôr na caixa (staging). git add . põe tudo o que mudou.

4. **git commit -m "…"**
   Fechar a caixa: um commit, com uma mensagem.

5. **git log --oneline**
   O histórico: um commit por linha, o mais recente em cima.

O CICLO: MUDAR, ADD, COMMIT

```
$ git init
Initialized empty Git repository

$ git status
Untracked files:
    livro.txt

$ git add livro.txt
$ git commit -m "Cria o livro"
[main (root-commit) 5c23e8e] Cria o livro

$ git log --oneline
5c23e8e (HEAD -> main) Cria o livro
```


FLAG

O livro: o primeiro repositório

1. Na pasta s01: mkdir livro, cd livro, git init e code .
2. Criar livro.txt (o título e uma frase) · git status · git add · git commit -m "Cria o livro"
3. Criar prefacio.txt · git status · add e commit -m "Acrescenta o prefácio"
4. Criar a pasta capitulos com capitulo1.txt · git add . · commit
5. Mudar o título no livro.txt · git status (modified) · add e commit
6. git log e git log --oneline: os quatro commits, os nossos nomes e as datas

RESUMO

O que escrevemos: o livro

GIT-S1/LIVRO

```
$ git init
$ git add livro.txt
$ git commit -m "Cria o livro"
$ git add prefacio.txt
$ git commit -m "Acrescenta o prefácio"
$ git add .
$ git commit -m "Acrescenta o capítulo 1"
$ git add livro.txt
$ git commit -m "Muda o título do livro"

$ git log --oneline
b4739da (HEAD -> main) Muda o título do livro
823da09 Acrescenta o capítulo 1
6402be1 Acrescenta o prefácio
5c23e8e Cria o livro
```

Um commit = uma ideia

Cada commit fechou uma coisa: o livro, o prefácio, o capítulo, o título.

O ritmo

Mudar → git status → git add → git commit. E outra vez.

FLAG

PARTE 4

Ver e desfazer

o que mudou, e como voltar atrás antes de estragar

DIFF → RESTORE → BOAS MENSAGENS → O EXERCÍCIO

FLAG

TEORIA

Ver o que mudou e desfazer

GIT DIFF: O QUE MUDOU (E AINDA NÃO ESTÁ GUARDADO)

```
$ git diff
--- a/capitulos/capitulo1.txt
+++ b/capitulos/capitulo1.txt
-Era uma vez alguém.
+Era uma vez alguém que queria ser developer.
```

Ler um diff

- a vermelho: a linha que saiu

+ a verde: a linha que entrou

Mudar uma linha = tirar a velha e pôr a nova.

O VS Code mostra o mesmo a cores (Source Control).

Quero ver…	
o que mudou e ainda não está na caixa	**git diff**
o que está na caixa (o próximo commit)	**git diff --staged**
Quero desfazer…	
as mudanças num ficheiro (volta ao último commit)	**git restore ficheiro**
um git add (tira da caixa; as mudanças ficam)	**git restore --staged ficheiro**
Quero apagar / mudar o nome de um ficheiro	**git rm · git mv**

FLAG

TEORIA

Boas mensagens de commit

Assim não

"alteração"
"teste"
"ultima"
"agora sim"
"mudanças várias no site todo"

Assim sim

"Acrescenta o prefácio"
"Corrige o link do menu para as sobremesas"
"Muda as cores do cabeçalho"
"Apaga as notas de rascunho"

1

Curta e no presente

"Acrescenta…", "Corrige…", "Muda…": completa a frase "este commit…". Até ~50 caracteres.

2

Um commit = uma ideia

Se a mensagem precisa de um "e", talvez sejam dois commits.

3

Para quem vem depois

Daqui a 3 meses, o "porquê" vai ser preciso. Esse alguém é quase sempre o próprio.

FLAG

No livro: ver, desfazer, apagar

1. Mudar uma frase no capitulo1.txt · git diff (o - e o +)
2. git add . · git diff (vazio!) · git diff --staged · commit
3. Estragar o prefacio.txt (apagar tudo e gravar) · git status · git restore prefacio.txt
4. Criar notas.txt · commit · depois git rm notas.txt · commit "Apaga as notas"
5. git log --oneline: sete commits

EXERCÍCIO

O site das receitas no Git

1. Copiar o vosso site das receitas para a vossa `s01/receitas` (ou o ponto de partida: `03-git-github/s01/receitas`)
2. Pôr o site no Git: o repositório e o primeiro commit, com o site todo
3. Fazer 3 mudanças pequenas no site, cada uma no seu commit, com uma boa mensagem
4. Estragar o `index.html` de propósito, ver o estrago com o diff e desfazê-lo
5. Apagar com o Git um ficheiro que já não é preciso (o `notas.txt`) e fazer o commit
6. No fim: colar o vosso `git log --oneline` no chat (pelo menos 5 commits)

Regras

Sem copiar comandos: usem o resumo (slide 25) e os snippets só se encravarem.

Na dúvida: git status.

Cerca de 25 minutos. Partilhas de ecrã às 11:55 e às 12:05.

FLAG

SOLUÇÃO

O site das receitas no Git

GIT-S1/RECEITAS

```
$ cd receitas
$ git init
$ git add .
$ git commit -m "Primeira versão do site das receitas"

# 3 mudanças, um commit cada (esta é uma)
$ git add css/estilos.css
$ git commit -m "Muda a cor do cabeçalho"

# o estrago: ver e desfazer
$ git diff index.html
$ git restore index.html

# apagar (ou: git mv antigo.css novo.css)
$ git rm notas.txt
$ git commit -m "Apaga as notas de rascunho"
$ git log --oneline
```

O que interessava

O ritmo status → add → commit; o diff antes do restore; mensagens que dizem o que mudou.

Os erros mais comuns

O git init na pasta errada (pwd!); o add esquecido ("nothing added to commit"); um restore sem querer, antes do commit.

FLAG

RESUMO DA SESSÃO

O que levamos de hoje

1

Controlo de versões

Guardar cada versão para voltar atrás, comparar e trabalhar em equipa. O Git é distribuído.

2

Uma vez

git config --global (nome, email, main, editor) por computador; git init por projeto.

3

Sempre

Mudar → git status → git add → git commit -m. E o git log --oneline para ver.

4

Antes de estragar

git diff para ver; git restore para desfazer; git rm e git mv para apagar ou mudar o nome.

A cábula

Os comandos de hoje estão todos nos snippets. A cábula oficial do GitHub (PDF): education.github.com/git-cheat-sheet-education.pdf

FLAG

ATÉ 6.ª FEIRA

Próxima sessão: branches e merge

1

Sessão 2 · 6.ª, 13 nov

Branches (experimentar sem estragar), o merge, os conflitos e o .gitignore. Voltar atrás no histórico.

2

Para casa (opcional)

O 3.2: a galeria responsiva (19.1) no Git, com um commit por passo. E acabar o 3.1, quem não acabou.

3

Para treinar

Learn Git Branching (learngitbranching.js.org, há em português): os primeiros níveis são o commit e o branch.

Snippets, enunciados e soluções: github.com/JoaoGoncalves/flag-frontend-2026 · pasta 03-git-github

FLAG

Formador

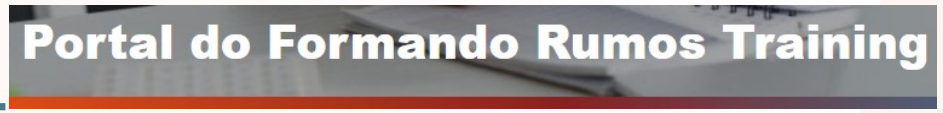
Caro(a) formador(a),

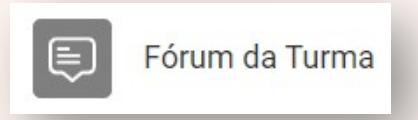
Recordamos que no final de cada sessão têm de garantir os registos inerentes ao curso:

Sumários – Módulo do Formador

Presenças - Módulo do Formador

Materiais – Moodle





FLAG

www.flag.pt

LISBOA Edifício Mirage, R. Dr. Eduardo Neves, Nº3, 1050-077 Lisboa email: querosabermais@flag.pt Tel. 213 560 606

PORTO Rua Oliveira Monteiro, Nº 168, 4050–438 Porto email: querosabermais@porto.flag.pt Tel. 226 094 789

uma marca

Rumos training | Knowledge sharing